LangChain Section 06

Memory & Conversation Management

LangChain v1.2.x (March 2026)

Section Roadmap

- Mental Model: Stateless LLMs and the memory problem
- Foundation: Message history as core memory mechanism
- RunnableWithMessageHistory: Adding memory to chains
- Backend storage: In-memory, Redis, PostgreSQL
- Strategies: Sliding window, summarization, token trimming
- Agents & persistence: LangGraph checkpointing
- Multi-session patterns: thread_id, user_id
- Long-term memory: Preferences and facts
- Common pitfalls and best practices

Note: LangChain v1.x deprecated ConversationBufferMemory. Modern approach: RunnableWithMessageHistory + trim_messages().

Mental Model: Why Memory Matters

The Stateless LLM Problem:

- LLMs process one prompt at a time
- No internal state between calls
- Each request is independent
- No memory of past conversations

Human Conversation Pattern:

- We retain context from earlier messages
- We reference prior statements
- We build on previous agreements
- Memory is essential for coherence

Core Insight

Memory bridges the gap: we manually track conversation history and inject it into each new prompt so the LLM can "see" the context.

The Stateless Problem in Action

Without Memory:

1. User: *"My name is Alice"*
2. LLM responds generically
3. User: *"What's my name?"*
4. LLM: *"I don't know, you didn't tell me."* ✗

With Memory:

1. User: *"My name is Alice"*
2. Store: `[User: My name is Alice]`
3. User: *"What's my name?"*
4. Inject history + new prompt
5. LLM: *"Your name is Alice."* ✓

Crucial

Without explicitly managing conversation history, each LLM call forgets everything.

Message History as Foundation

Message History

A list of messages (user, assistant, system) that represents the conversation. This is the **fundamental building block** of memory in LangChain.

Message Types:

- `HumanMessage`: User input
- `AIMessage`: LLM response
- `SystemMessage`: Instructions/context
- `ToolMessage`: Tool output
- `ToolCall`: Function invocation record

Example Structure:

Message List

messages = [SystemMessage(content="You are helpful"), HumanMessage(content="My name is Alice"), AIMessage(content="Nice to meet you!")]

Manual Message History Management

The basic pattern: append and pass

Basic Loop

```
messages = []
user_input = ‘‘My name is Alice’’
messages.append(HumanMessage(content=user_input))
response = model.invoke(messages)
messages.append(AIMessage(content=response.content))
```

Key points:

- Each turn: append user message
- Invoke model with full message history
- Append response to history
- Next call sees all prior context

[Docs: langchain.schema.messages]

Problems with Manual Management

Scalability Issues

- Messages list grows indefinitely
- Context window gets consumed
- Expensive API calls (charged per token)
- Requires manual bookkeeping in production

Solution Direction

LangChain provides abstractions to:

- Automatically store/retrieve history
- Trim old messages strategically
- Use persistent backends
- Manage multiple conversations

RunnableWithMessageHistory: Adding Memory to Chains

The Modern Approach: Wrap a chain with automatic history management

Setup

```
chain = prompt | model
chain_with_history = RunnableWithMessageHistory(
chain,
get_session_history=func,
input_messages_key=``input''
)
```

Usage

```
response = chain_with_history.invoke(
{``input'': ``What's my name?''},
config={``configurable'': {``session_id'': ``user_123''}}
)
```

Implementing get_session_history

The callback that retrieves/stores history for a session:

In-Memory Backend

```
store = {}

def get_session_history(session_id: str):
if session_id not in store:
store[session_id] = ChatMessageHistory()
return store[session_id]
```

Or use persistent backends (PostgreSQL, Redis, etc.) [Docs: BaseChatMessageHistory]

Chat Message History Backends

Where conversation history is stored:

Backend	Use Case	Persistence
ChatMessageHistory	Testing, demos	Memory only
RedisChatMessageHistory	Real-time, high throughput	Redis
PostgresChatMessageHistory	Production, long-term	Database
DynamoChatMessageHistory	AWS deployments	DynamoDB
FileChatMessageHistory	Local files, development	Filesystem

Choice Guide

Development: ChatMessageHistory or FileChatMessageHistory
Production: PostgreSQL or Redis for scale and persistence

Redis Chat History Example

Redis Backend

```
def get_session_history(session_id: str):
return RedisChatMessageHistory(
session_id=session_id,
redis_client=redis.Redis(
host='localhost'
),
ttl=86400
)
```

TTL Feature

Time-To-Live: messages automatically expire after 24 hours

[Docs: RedisChatMessageHistory]

Sliding Window Memory: trim_messages()

Keep only recent messages to control context window:

Trimming Configuration

```
trimmer = trim_messages(
max_tokens=2000,
strategy=``last'',
include_system=True,
allow_partial=False
)
```

Effect

Prevents unbounded growth of conversation history while preserving recent context.

[Docs: trim_messages]

Summary Memory: Condensing Old Conversations

Strategy: Periodically summarize older messages

Before:

- 100+ messages
- Full context bloat
- Expensive tokens

After:

- Summary of old discussion
- Recent messages in full
- Compressed context

Pattern:

1. Detect when history exceeds 50 messages
2. Call LLM to summarize older messages
3. Replace with SystemMessage containing summary
4. Keep recent messages as full history

Token-Based Trimming

Advanced: Trim by token count, not message count

Token Counting

```
trimmer = trim_messages(
max_tokens=2000,
strategy=''last'',
include_system=True
)
trimmed = trimmer.invoke(messages)
```

Advantage

Respects actual token budget, not message count. Most accurate for cost control.

[Docs: trim_messages]

MessagesPlaceholder in Prompts

Inject conversation history into prompt templates:

Prompt Template

```
prompt = ChatPromptTemplate.from_messages([
(``system'', ``You are helpful''),
MessagesPlaceholder(``chat_history''),
(``user'', ``{input}'')
])
```

How It Works

MessagesPlaceholder expands to the full message list at runtime.

[Docs: MessagesPlaceholder]

Memory in Agents: LangGraph Checkpointing

For agents with tools, use LangGraph persistence:

LangGraph Advantages:

- Automatic state saving
- Tool call history
- Thread-based sessions
- Step-by-step replay

Built-in Persistence:

- MemorySaver (in-memory)
- PostgresSaver (production)
- Custom backends

Thread ID Pattern

Each user/session gets a `thread_id`. LangGraph tracks all state within that thread.

LangGraph Agent Memory Example

Graph with Memory

```
memory = MemorySaver()
agent_graph = builder.compile(checkpointer=memory)

config = {``configurable'': {``thread_id'': ``user_123''}}

response = agent_graph.invoke(
{``messages'': [HumanMessage(``2+2?'')]},
config=config
)
```

Next invocation: state automatically restored from checkpoint. [Docs: langgraph.checkpoint]

PostgreSQL Persistence for Production

PostgreSQL Checkpointer

```
postgres_saver = PostgresSaver.from_conn_string(
``postgresql://user:pwd@localhost/db''
)

agent_graph = builder.compile(
checkpointer=postgres_saver
)
```

Persistence

Each `thread_id` persists across application restarts via PostgreSQL.

[Docs: langgraph.checkpoint.postgres]

Multi-Session Memory: thread_id & user_id

Patterns for managing multiple concurrent conversations:

thread_id (LangGraph):

- Unique per conversation
- Identifies a message thread
- Manages agent state
- Built-in replay capability

session_id (Runnable):

- Unique per user/chat
- Retrieves history
- Backend agnostic
- Simple key-value lookup

Best Practice

Use format: `thread_id = user_id + ‘‘_’’ + conversation_id`

Multi-User Session Management

Session Handling

```
def handle_chat(user_id: str, message: str):
session_id = user_id + ‘‘_’’ + str(uuid.uuid4())

response = chain_with_history.invoke(
{‘‘input’’: message},
config={‘‘configurable’’:
{‘‘session_id’’: session_id}}
)
return response
```

Always include `user_id` in session identifier to prevent cross-user leakage.

Long-Term Memory: Preferences & Facts

Beyond message history: storing extracted knowledge

User Preferences:

- Communication style
- Topics of interest
- Timezone, language
- Communication frequency

Extracted Facts:

- Name, role, department
- Project assignments
- Known constraints
- Previous decisions

Pattern: Extract facts from conversations, store in database, inject into prompts.

Extracting & Storing Long-Term Facts

Fact Extraction

```
class UserFact(BaseModel):
key: str
value: str

extraction = model.with_structured_output(UserFact)
facts = extraction.invoke({''messages'': messages})
```

Storage

Store facts in database table: user_facts(user_id, key, value). Inject on session start.

Injecting Long-Term Facts into Prompts

Fact Injection

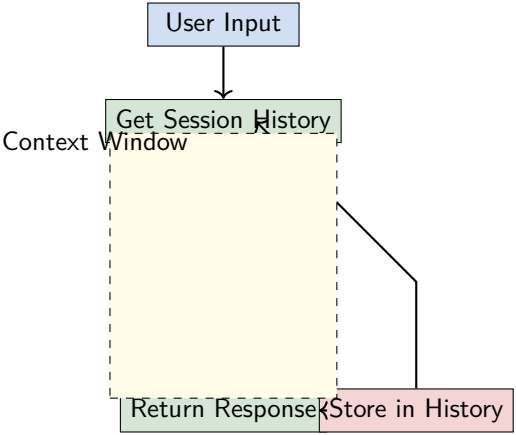
```
facts = db.query(user_facts)
facts_text = ``''.join(
[``- '' + f.key + ``:  ''  + f.value
for f in facts]
)

system_prompt = ``You are helpful.''  + facts_text
```

Effect

Personalizes every response without re-querying conversation history.

TikZ: Memory Flow in Conversation



Comparison Table: Memory Strategies

Strategy	Simple	Efficient	Safe	Best For
Buffer (no trim)	✓		Risk	Short sessions
Sliding Window	✓	✓	Good	Most use cases
Summary		✓	Possible	Long conversations
Hybrid		✓	Good	Production
Long-term Facts	✓	✓	Good	Personalization

Recommendation

Start: Sliding window + MessagesPlaceholder
Scale: Hybrid (window + summary) + long-term facts database

Common Pitfall 1: Unbounded Context Growth

Problem

Conversations grow without limit:

- 10 messages $\rightarrow$ 100 messages $\rightarrow$ 10,000 messages
- Context window exhausted
- LLM starts ignoring early context
- Expensive API calls (charged per token)

Solution

Always use `trim_messages()` or sliding window strategy.

- Set `max_tokens` budget
- Monitor conversation length
- Archive old conversations

Common Pitfall 2: Session Confusion

Problem

Multi-user systems mixing conversations:

- User A sees User B's history
- Same session_id used for different users
- Database race conditions

Solution

- Always include user_id in session_id
- Format: `session_id = user_id + ‘‘_’’ + conv_id`
- Test with multiple concurrent users
- Audit session history retrieval

Common Pitfall 3: Forgetting to Trim

Symptom

LLM responses become short or irrelevant after 50+ messages

Root Causes:

- No trimmer configured
- Trimmer not in message processing pipeline
- Token budget too small
- System message getting dropped

Prevention Checklist

- ✓Create trimmer with appropriate max_tokens
- ✓Add to chain: `chain = prompt | trimmer | model`
- ✓Always set `include_system=True`
- ✓Test with 100+ message conversations

Common Pitfall 4: Losing Important Context

Issue

Trimming too aggressively causes context loss:

- Critical earlier agreements forgotten
- Repeated explanations needed
- User frustration

Mitigation Strategies

- Use summary memory for 50+ message conversations
- Store key decisions in long-term facts database
- Set `strategy=''last''` to keep recent messages
- Hybrid approach: summary + last 10 messages

Production Checklist

1. **Choose Backend:** PostgreSQL for persistence, Redis for speed
2. **Set Memory Budget:** Define max_tokens for conversations
3. **Implement Trimming:** Use trim_messages() with strategy
4. **Multi-Session:** Use user_id + conversation_id patterns
5. **Extract Facts:** Periodically extract and store user facts
6. **Test Thoroughly:** Simulate long conversations, concurrent users
7. **Monitor Costs:** Track token usage per session
8. **Archive Old:** Periodically move old sessions to cold storage
9. **Audit Access:** Ensure users only see their own history

Summary

Core Concepts:

- LLMs are stateless; memory is essential
- Message history is the foundation
- RunnableWithMessageHistory automates storage/retrieval
- trim_messages() controls context growth
- LangGraph provides agent state persistence

Modern Patterns:

- Sliding window memory for most cases
- Summary memory for long conversations
- Long-term facts for personalization
- PostgreSQL backend for production

Remember

Memory is not a luxury; it's fundamental to building coherent, contextual applications.

Cheat Sheet: Quick Reference

Core Imports:

Imports

```
from langchain.runnables.history import RunnableWithMessageHistory
from langchain_text_splitters import trim_messages
from langchain_postgres import PostgresChatMessageHistory
from langgraph.checkpoint.postgres import PostgresSaver
```

Basic Setup:

1. Create chain: `chain = prompt | model`
2. Implement `get_session_history(session_id)`
3. Wrap: `RunnableWithMessageHistory(chain, get_session_history, ...)`
4. Use: `chain.invoke({...}, config={``configurable'': {``session_id'': ...}})`

Self-Check Questions (1-4)

1. **Why do LLMs need external memory?** *Answer: LLMs are stateless and process one prompt at a time with no built-in memory.*
2. **What does trim_messages() do?** *Answer: Removes old messages to keep conversations within a token budget.*
3. **What is the difference between session_id and thread_id?** *Answer: session_id is for RunnableWithMessageHistory (simple history), thread_id is for LangGraph agents (full state).*
4. **How do you prevent session confusion in multi-user systems?** *Answer: Always include user_id in the session identifier.*

Self-Check Questions (5-8)

5. **When should you use summary memory?** *Answer: When conversations exceed 50+ messages and token budget is tight.*
6. **What is MessagesPlaceholder used for?** *Answer: Injecting the full conversation history into a prompt template at runtime.*
7. **Name two chat history backends besides in-memory.** *Answer: PostgreSQL (PostgresChatMessageHistory) and Redis (RedisChatMessageHistory).*
8. **How do you extract long-term facts from conversations?** *Answer: Use structured output extraction with Pydantic models, then store in database.*

Further Resources

- **LangChain Docs:** https://python.langchain.com/docs/modules/memory/
- **RunnableWithMessageHistory:** https://python.langchain.com/docs/how_to/message_history
- **LangGraph:** https://langchain-ai.github.io/langgraph/
- **trim_messages:** https://python.langchain.com/docs/how_to/trim_messages
- **Chat History Backends:** https://python.langchain.com/docs/integrations/memory

Next Steps

Experiment with different memory strategies on a small project. Monitor token usage and adjust max_tokens for your use case.

Key Takeaways

- **1.** Memory is explicit in LangChain; you control how it works
- **2.** Message history is the foundation; everything builds on it
- **3.** RunnableWithMessageHistory handles automatic storage
- **4.** Always trim to prevent unbounded context growth
- **5.** Use persistent backends (PostgreSQL) in production
- **6.** Extract and store long-term facts separately
- **7.** LangGraph provides agent state persistence
- **8.** Test multi-user scenarios carefully

End of Section 06

Memory & Conversation Management

You now understand how to build stateful, context-aware LLM applications.

Next: Section 07 — Agents & Tools